Submission Catalogue

Submission Catalogue **1**
Note on Evaluation: 1
1. Overview **2**
2. UML Class Diagram **3**
3. Written Scenarios **4**
Scenario 1: Player Uses an Item to Defeat a Monster 4
Scenario 2: Player Manages Inventory Weight to Progress 4
4. Object Diagrams **6**
Scenario 1: Player Uses an Item to Defeat a Monster 6
5. Sequence Diagrams **10**
Scenario 1: Player Uses an Item to Defeat a Monster 10
Scenario 2: Player Manages Inventory Weight to Progress 12

Note on Evaluation:

For Homework 8, the **Overview**, **UML diagrams** and **written scenarios** have been refined to reflect the actual implementation. The first two entries in each section (Written Scenarios, Object Diagrams, and Sequence Diagrams) have been updated to align with the implemented codebase and are designated as primary for grading. The UML Class Diagram has been fully updated to reflect the final system architecture.

1. Overview

The system implements a data-driven text-adventure game engine following the **Model-View-Controller (MVC)** architectural pattern. The engine loads game content from external JSON files at runtime via **JsonGameLoader**, enabling flexible game creation without modifying source code. The core design emphasizes a strict separation between the game logic (**Model**), user interaction handling (**Controller**), and application bootstrapping (**Engine Driver**).

The **Model** layer encapsulates all game state and logic. At its center is the **IGameModel** interface, which defines the contract between the Controller and the Model — the Controller never directly accesses internal classes such as **Player**, **Room**, or **Item**. The concrete **GameModel** implements this interface and orchestrates movement, inventory management, puzzle solving, monster encounters, and scoring. All methods return String messages, ensuring the Model performs no I/O.

The **Player** tracks health (0–100), a cumulative score, current room number, and an **Inventory** with a weight limit of 13. Health is categorized by the **HealthStatus** enum (AWAKE, FATIGUED, WOOZY, SLEEP); the game ends when health reaches zero. The **GameWorld** serves as a centralized registry for all **Room**, **Item**, **Fixture**, **Puzzle**, and **Monster** objects. Rooms are connected via directional exits modeled by the **Direction** enum, where positive values indicate passable paths, zero indicates walls, and negative values indicate exits blocked by an active puzzle or monster.

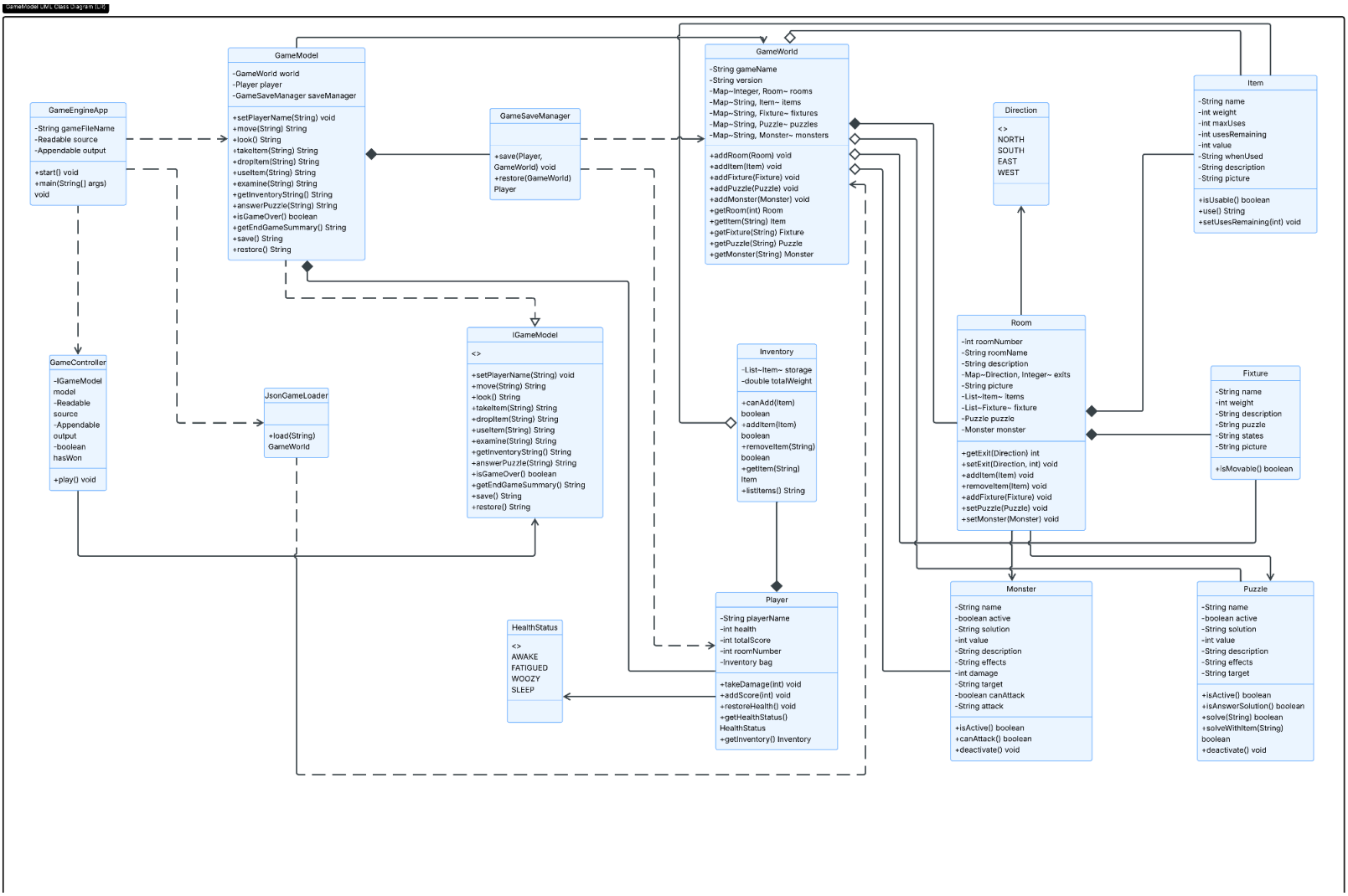
Items have limited uses and weight; players must make strategic carry decisions. **Puzzles** gate blocked exits and are solved via text answers or item usage. **Monsters** block exits and deal damage on movement attempts, defeated by using the correct item. Solving a puzzle or defeating a monster deactivates it, opens blocked exits, and awards score. **Fixtures** are immovable objects that provide narrative clues when examined.

The **Controller** (**GameController**) manages the game loop — reading input from a Readable, dispatching commands to **IGameModel**, and writing output to an Appendable. Supported commands include directional movement, looking, inventory operations (take, drop, use, examine), puzzle answering, and save/restore. The **GameSaveManager** provides persistence by serializing player stats, inventory, item usage, puzzle/monster states, and room exits to a JSON file. **GameEngineApp** serves as the entry point, wiring the loader, model, and controller together.

Overall, the design enforces strict separation of concerns: the **Model** owns all logic, the **Controller** handles I/O, and the data-driven architecture ensures game content is independent of engine code — each layer can be tested and maintained in isolation.

2. UML Class Diagram

This diagram provides a complete overview of all classes and relationships in the system. This diagram provides a complete overview of all classes and relationships in the system.



3. Written Scenarios

Scenario 1: Player Uses an Item to Defeat a Monster

As the player explores the game world, certain rooms contain an active Monster. Upon entering such a room, the game engine immediately displays the monster's effects text along with a warning that the monster is blocking the player's path. No damage is dealt simply by entering the room.

If the player attempts to move in a direction that is blocked (negative exit value) while a monster is present and its can_attack flag is true, the monster attacks the player. The game engine applies the monster's damage value to the player's health and displays the attack message. The player's updated health and health status are shown after each attack. This continues every time the player attempts to push through the blocked exit.

To defeat the monster, the player must have the correct item in their inventory — specifically, the item whose name matches the monster's solution field. The player issues a Use command with that item's name. The game engine decrements the item's remaining uses, applies the use effect, then checks whether the item matches the monster's solution. If it matches, the monster transitions from active to defeated, all negative exits in that room are flipped to positive, and the player is awarded the monster's point value. If the item runs out of uses in the process, it is automatically removed from the player's inventory.

Key entities involved: Player, Room, Monster, Item, Inventory, exit, health, score

---

Scenario 2: Player Manages Inventory Weight to Progress

As the player explores rooms, items are scattered throughout with varying weights. Each item's name and weight are displayed when the player looks at a room.

When the player issues a Take command for an item, the game engine checks whether adding that item's weight to the current inventory total would exceed the capacity limit. If it would, the action fails and a message is returned explaining the weight limit. The item remains in the room.

To pick up a heavier item, the player may first drop one or more items from their inventory into the current room using the Drop command. Dropped items remain in that room and can be retrieved later. After freeing up sufficient capacity, the player can successfully take the desired item.

Since some puzzles and monsters require specific items as their solution, careful inventory management — deciding which items to carry and which to leave behind — is an essential part of progressing through the game.

Key entities involved: Player, Inventory, Item, weight, capacity, Room, takeItem, dropItem

4. Object Diagrams

Scenario 1: Player Uses an Item to Defeat a Monster

Context: The Player has entered the Foyer but cannot proceed East because the path is blocked by the Teddy Bear monster. The player must use the Hair Clippers item to defeat it and unlock the exit.

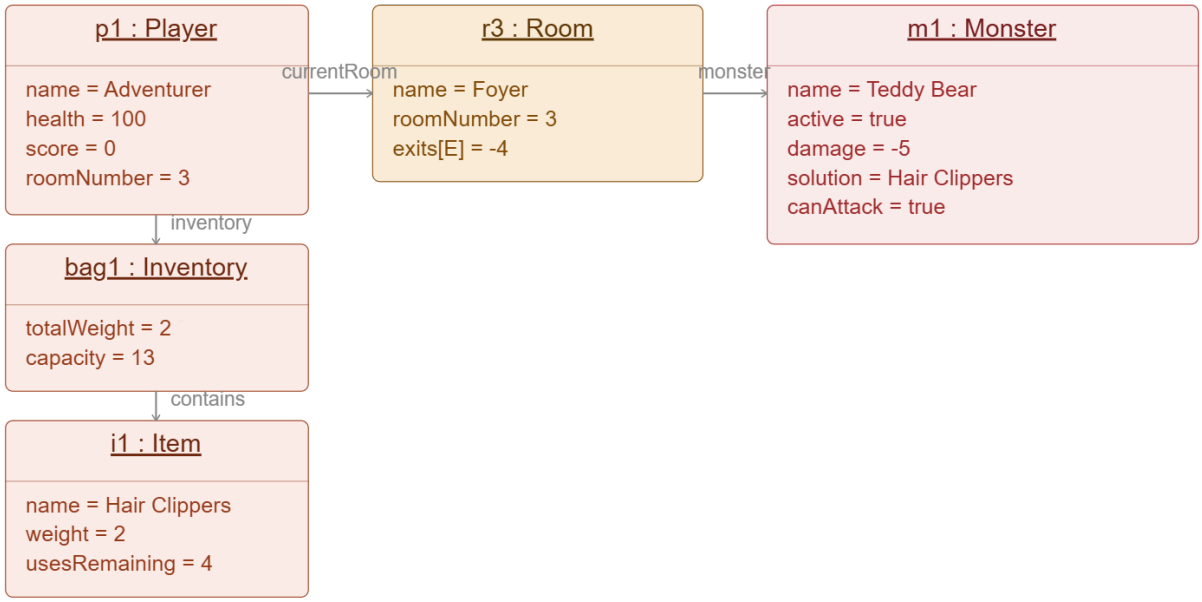
Snapshot: The "Monster Blocking" State

Object Instance	Class	Attributes/State
p1	Player	name="Adventurer", health=100, score=0, roomNumber=3
bag1	Inventory	totalWeight=2, capacity=13
r3	Room	name="Foyer", roomNumber=3, exits[E]=-4
m1	Monster	name="Teddy Bear", active=true, damage=-5, solution="Hair Clippers", canAttack=true
i1	Item	name="Hair Clippers", weight=2, usesRemaining=4, value=5

Logics:

- `p1` is in `r3` (currentRoom)
- `p1` holds `bag1` (Inventory)
- `bag1` contains `i1` (Hair Clippers)
- `m1` is in `r3` (monster blocks east exit)
- `r3`.exits[E] = -4 (blocked by `m1`)
- `p1` attempts move E → blocked, `m1` attacks → `p1`.health = 95
- `p1` issues U Hair Clippers → `i1`.usesRemaining = 3
- `i1` matches `m1`.solution → `m1`.active = false
- `r3`.exits[E] flips to 4 (unlocked)
- `p1`.score = 200

Snapshot: The "Monster Blocking" State



Scenario 2: Player Manages Inventory Weight to Progress

Context: The Player is in the Giant Plate room and wants to take the Carrot, but their inventory is too heavy. They must drop the Lamp first before they can pick up the Carrot.

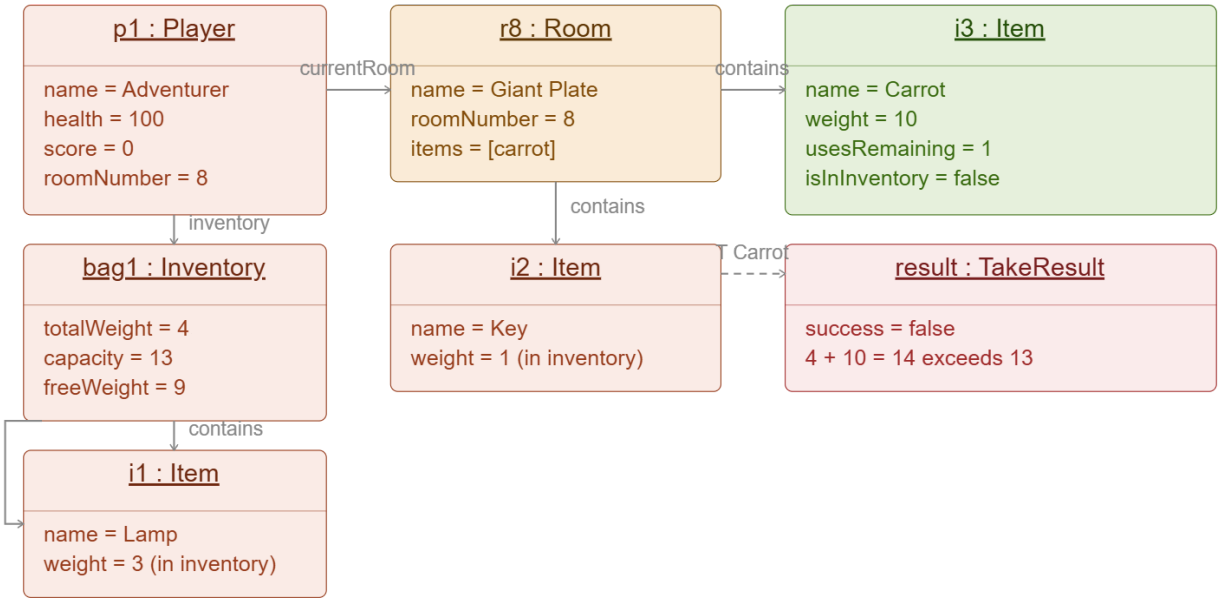
Snapshot: The "Overweight" State

Object Instance	Class	Attributes/State
p1	Player	name="Adventurer", health=100, score=0, roomNumber=8
bag1	Inventory	totalWeight=4, capacity=13, freeWeight=9
r8	Room	name="Giant Plate", roomNumber=8, items=[carrot]
i1	Item	name="Lamp", weight=3, usesRemaining=20, value=100, isInInventory=true
i2	Item	name="Key", weight=1, usesRemaining=3, value=5, isInInventory=true
i3	Item	name="Carrot", weight=10, usesRemaining=1, value=5, isInInventory=false

Logics:

- p1 is in r8 (current location)
- p1 holds bag1 (Inventory)
- bag1 contains i1 (Lamp, weight=3)
- bag1 contains i2 (Key, weight=1)
- i3 is in r8(room contains Carrot)
- p1 attempts T Carrot → 4+10=14 > 13 → **fails**
- p1 issues D Lamp → bag1.totalWeight = 1, i1 moves to r8
- p1 attempts T Carrot → 1+10=11 ≤ 13 → **succeeds**
- bag1.totalWeight = 11 after successful take

Snapshot: The "Overweight" State



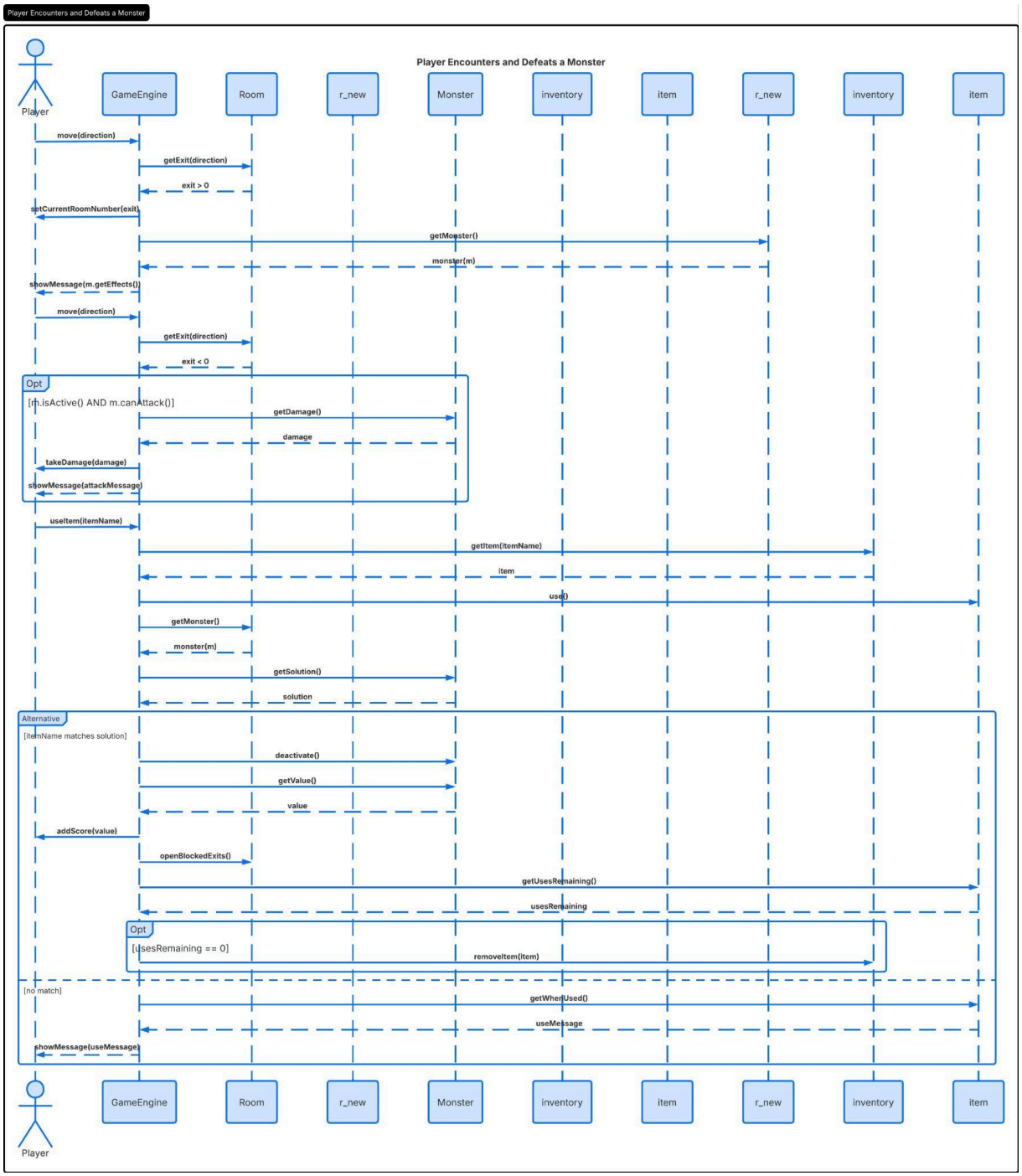
5. Sequence Diagrams

Scenario 1: Player Uses an Item to Defeat a Monster

Logics:

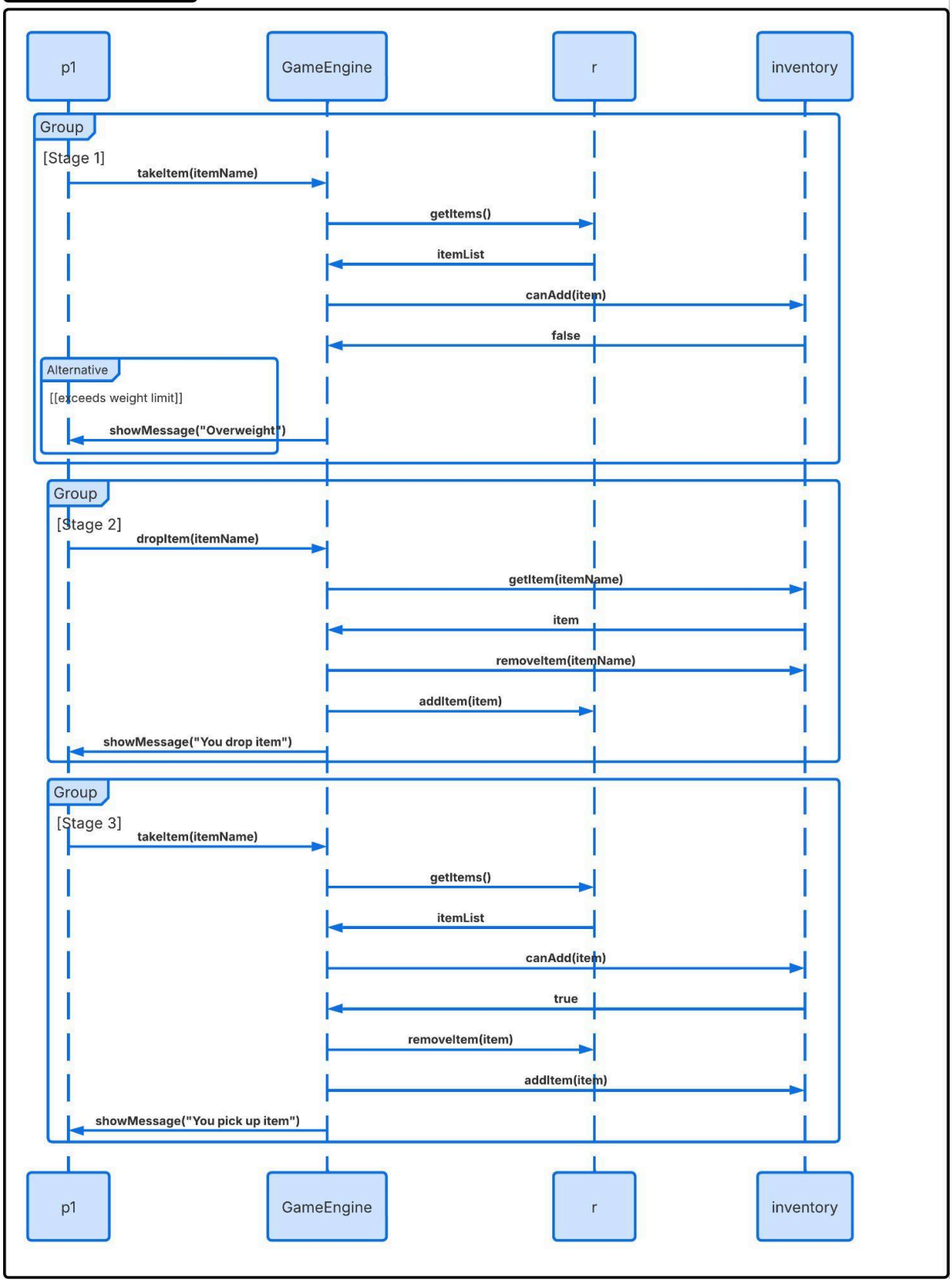
```
// Stage 1: Go into a room has a monster
p1 → GameEngine : move(direction)
GameEngine → r : getExit(direction)
r → GameEngine : exit > 0
GameEngine → p1 : setCurrentRoomNumber(exit)
GameEngine → r_new : getMonster()
r_new → GameEngine : monster(m)
GameEngine → p1 : showMessage(m.getEffects())


// Stage 2: Player tries to go to blocked exit
p1 → GameEngine : move(direction)
GameEngine → r : getExit(direction)
r → GameEngine : exit < 0

opt [m.isActive() AND m.canAttack()]
   GameEngine → p1 : takeDamage(m.getDamage())
   GameEngine → p1 : showMessage(attackMessage)
end


// Stage 3: Player defeats the monster by using item
p1 → GameEngine : useItem(itemName)
GameEngine → inventory : getItem(itemName)
inventory → GameEngine : item

GameEngine → item : use()
GameEngine → r : getMonster()
r → GameEngine : monster(m)

alt [itemName matches m.getSolution()]
   GameEngine → m : deactivate()
   GameEngine → p1 : addScore(m.getValue())
   GameEngine → r : openBlockedExits()

   opt [item.usesRemaining == 0]
```

```
        GameEngine → inventory : removeItem(item)
    end

else [no match]
    GameEngine → p1 : showMessage(item.getWhenUsed())
end
```



Scenario 2: Player Manages Inventory Weight to Progress

Logics:

```
// Stage 1: Player tries to get item(failed)
p1 → GameEngine : takeItem(itemName)
GameEngine → r : getItems()
r → GameEngine : itemList
GameEngine → inventory : canAdd(item)
inventory → GameEngine : false

alt [exceeds weight limit]
   GameEngine → p1 : showMessage(“Overweight")
end


// Stage 2: Player drops items
p1 → GameEngine : dropItem(itemName)
GameEngine → inventory : getItem(itemName)
inventory → GameEngine : item
GameEngine → inventory : removeItem(itemName)
GameEngine → r : addItem(item)
GameEngine → p1 : showMessage("You drop item")

// Stage 3: Player get the item
p1 → GameEngine : takeItem(itemName)
GameEngine → r : getItems()
r → GameEngine : itemList
GameEngine → inventory : canAdd(item)
inventory → GameEngine : true
GameEngine → r : removeItem(item)
GameEngine → inventory : addItem(item)
GameEngine → p1 : showMessage("You pick up item")
```

Item Pickup/Drop Flow (3 Stages)